COURSEWORK ASSIGNMENT SUBMISSION

Name: Daksh Srivastava **Roll No:** 150096725087
Subject: Software Engineering & Project Management **Day:** Day 8 — Software Engineering Principles & Characteristics

---

Section 8: Classroom Activity — Diagnose a Poorly Designed System

Diagnostic analysis of an anti-pattern system where one monolithic script handles authentication, event cataloging, seat capacity checking, registration, email alerts, and database transactions.

1. Violated Software Engineering Principles

- **Single Responsibility Principle (SRP):** The monolithic component has multiple reasons to change—database connection modifications, email template edits, or authentication logic changes all alter the identical code file.
- **Don't Repeat Yourself (DRY):** The capacity validation logic (`if (count >= max_capacity)`) is manually duplicated across three separate screens, guaranteeing defect divergence when the capacity rule is updated in one location but forgotten in another.
- **Separation of Concerns & Modularity:** Presentation logic (UI screens), business rules (capacity and conflict checks), and persistence logic (raw SQL database queries) are completely entangled.

2. Operational Impact of Duplicated Capacity Rules

If event organizers introduce a 10% overflow waitlist policy, the engineering team must locate and update the logic across all three screens. If one screen is missed, the system will reject waitlisted students on mobile web while accepting them on desktop, leading to data corruption and student disputes.

3. Quality Attributes at Severe Risk

- **Maintainability:** Extremely high cognitive load; minor bug fixes risk introducing unexpected side-effects across unrelated features.
- **Testability:** Zero automated testability because business rules cannot be unit-tested without instantiating UI layouts and live database connections.
- **Reliability:** An unhandled exception in the email notification dispatch routine crashes the entire thread, aborting the database registration transaction.

4. Proposed Architectural Improvements

- **Improvement 1 (Modular Service Layer):** Extract business rules into a dedicated `RegistrationService` that encapsulates capacity validation as a single authoritative method.
- **Improvement 2 (Asynchronous Notification Queue):** Decouple email notifications from registration transactions using an asynchronous background queue (RabbitMQ / Redis BullMQ).
- **Improvement 3 (Automated Unit & Integration Test Suite):** Implement automated unit tests for capacity validation logic and integration tests asserting database rollbacks on failure.

Section 12: Homework — CEMS Principles Consolidation Table

Structured architectural specification synthesizing functional rules, quality standards, and design rationale for CEMS.

Category	Item ID & Title	Detailed Specification	Engineering Rationale / Target
Functional	FR-01: Event Filtering	Students can filter events by technical domain and date range.	Reduces cognitive overload; enables rapid discovery.
Functional	FR-02: Seat Reservation	System reserves seat atomically upon valid form submission.	Prevents over-allocation of limited laboratory venues.
Functional	FR-03: Self-Cancellation	Students can cancel registration up to 24 hrs prior to event.	Recovers unused seats for waitlisted students.
Functional	FR-04: Gate Check-in	Volunteers can mark attendance by scanning student QR tokens.	Eliminates entry bottleneck queues at lecture halls.
Functional	FR-05: Attendance Export	Organizers can export signed attendance sheets in CSV/PDF.	Mandatory for student extracurricular credit filing.
Non-Functional	NFR-01: Performance	p95 API response time under 1.5 seconds at 300 req/sec.	Prevents portal crash during 10:00 AM rush hour.
Non-Functional	NFR-02: Security	All passwords hashed using Argon2id / BCrypt (salt factor 12).	Protects student credentials against data leaks.
Non-Functional	NFR-03: Availability	99.5% operational uptime during active registration period.	Ensures equal registration access for all students.
Architecture	DEC-01: RESTful Services	Decouple React SPA frontend from Express API backend.	Allows independent scaling and frontend redesign.
Architecture	DEC-02: Transaction Locks	Use PostgreSQL row-level locks for registration commits.	Guarantees zero overbooking under race conditions.
Architecture	DEC-03: Edge Static Caching	Serve images, CSS, and static event data via Cloudflare CDN.	Offloads 70% of inbound bandwidth from origin server.